

HW5_CNN_vs_digits

December 26, 2024

0.0.1 Data analysis in Astronomy

0.0.2 HW5 Classification of digits

0.0.3 Deadline 12/26 23:59

0.0.4 Your name:

0.0.5 Again, print out your code and the results and the final plot into pdf.

0.0.6 Challenge: build a CNN model to classify images with digits and obtain the final accuracy for the test dataset > 0.9912 (the record we get from the kernel SVM)

If your model can obtain the accuracy of the test dataset > 0.9912 , you get 100/100.

If your model can obtain the accuracy of the test dataset between 0.99 and 0.9912, you get 90/100.

If your model can obtain the accuracy of the test dataset between 0.985 and 0.99, you get 80/100.

You have to at least get > 0.985 in order to get the points. But, it should not be difficult. (It took me ~15 mins to find a model that can do 0.997 accuracy of the test data.)

0.0.7 This is a bonus HW.

Have fun!

```
[46]: from google.colab import drive
drive.mount('/content/gdrive', force_remount=True)
root_dir = "/content/gdrive/My Drive/Colab Notebooks/"

import astropy.io.fits as pf
import numpy as np
import matplotlib.pyplot as plt
import torch
from skimage.transform import rotate
```

Mounted at /content/gdrive

```
[47]: '''A function to convert the image data into Pytorch tensor matrix'''

def give_me_Pytorch_data_form(input_image, input_labels, torch_format=True):
    whole_length = len(input_image[0])
```

```

width = int(np.sqrt(whole_length))
image_matrix = np.zeros((len(input_image),1,width,width))
labels_matrix = np.zeros((len(input_image),10))
for i in range(0,len(input_image)):
    image_matrix[i,0,:,:]=input_image[i].reshape(width,width)
    labels_matrix[i,input_labels[i]]=1.

if torch_format==True:
    image_matrix = torch.from_numpy(np.array(image_matrix,dtype=np.float32))
    labels_matrix = torch.from_numpy(np.array(labels_matrix,dtype=np.
↪float32))

return image_matrix,labels_matrix

```

Training data https://drive.google.com/file/d/1UQufnFEJFpJR3-fFjImm_oXVTsSJ5KrA/view?usp=sharing

Test data https://drive.google.com/file/d/1r2yuYPKoY386PqWfz8rf-JsmgE4_lmHt/view?usp=sharing

```

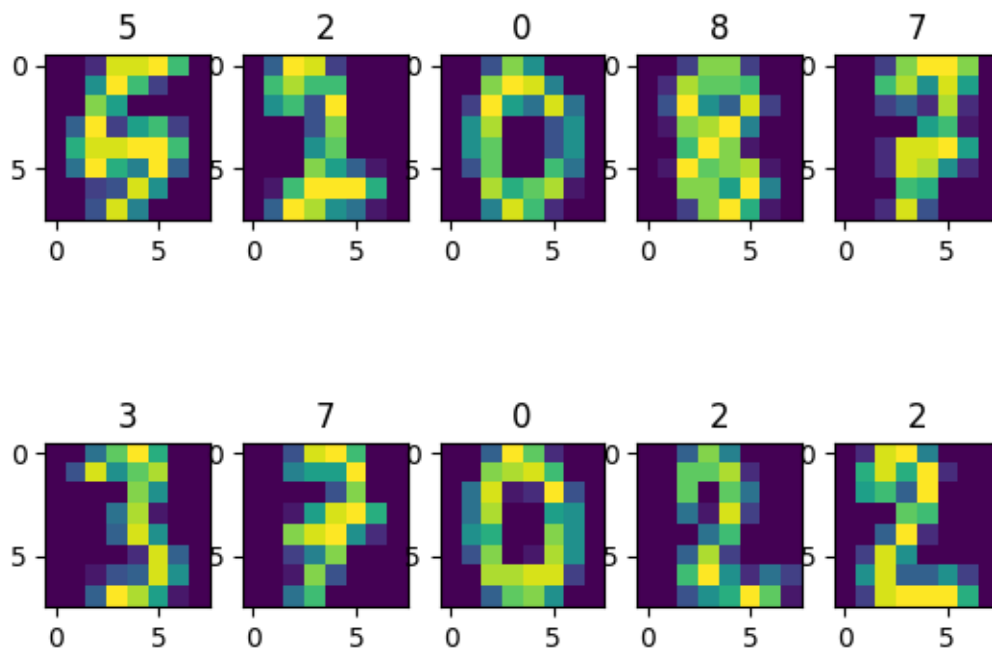
[48]: train = pf.open(root_dir+'Training.fits')
      test = pf.open(root_dir+'Test.fits')

```

```

[49]: for i in range(0,10):
      plt.subplot(2,5,i+1)
      plt.imshow(train[0].data[i].reshape(8,8))
      plt.title(train[1].data[i])

```



```
[50]: '''Converting original data into Pytorch data'''
image_train, image_labels = give_me_Pytorch_data_form(train[0].data,train[1].
↳data,torch_format=True)
image_test, test_labels = give_me_Pytorch_data_form(test[0].data,test[1].data)
```

```
[51]: '''A simple CNN architecture'''
'''You need to upgrade it to get better performance'''

class CNN(torch.nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        ###
        ### input 1x8x8
        self.cnn1 = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=1,↳
↳out_channels=8,kernel_size=3,padding=1,stride=1), #8x8x8
            torch.nn.BatchNorm2d(8),
            torch.nn.ReLU(),
        )

        self.fc1 = torch.nn.Sequential(
            torch.nn.Linear(512, 10))

    def forward(self, x):
        x = self.cnn1(x)
        x = x.view(x.size(0), -1)
        x = self.fc1(x)
        out = x
        return out
```

```
[52]: model = CNN()
```

```
[53]: learningRate = 0.01
epochs = 100
criterion = torch.nn.CrossEntropyLoss()
# Just the loss function : here we use the default CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learningRate)
```

```
[54]: accuracy_array = []
accuracy_test_array = []
loss_array = []
loss_test_array = []
```

```

epoch_array = []
N_total_train = len(image_train)
batch_size=64
for epoch in range(epochs):

    model.train() # here is to tell the code to do the training again after
    ↪model.eval() mode
    # Clear gradient buffers because we don't want any gradient from previous
    ↪epoch to carry forward, dont want to cumulate gradients
    for start_index_batch in range(0,N_total_train,batch_size):
        # Clear gradient buffers because we don't want any gradient from
        ↪previous epoch to carry forward, dont want to cumulate gradients
        optimizer.zero_grad()
        end_index = min(start_index_batch + batch_size, N_total_train)
        # get output from the model, given the inputs
        #print(len(color_train[start_index_batch:end_index,:]))
        outputs = model(image_train[start_index_batch:end_index])

        # get loss for the predicted output
        loss = criterion(outputs, image_labels[start_index_batch:end_index,:])
        # get gradients w.r.t to parameters
        loss.backward()
        # update parameters
        optimizer.step()

    model.eval() # here is to fix the parameters (mean, sigma) of the
    ↪batchnormalization, given that the batchnorm calculates the mean sigma with
    ↪a new batch everytime during training.
    with torch.no_grad():
        outputs_all = model(image_train)
        pred_y = torch.max(outputs_all, 1)[1].data.squeeze()
        accuracy = torch.sum((pred_y == torch.max(image_labels, 1)[1].data.
        ↪squeeze()) / pred_y.size(0))
        epoch_array.append(epoch)

    loss_array.append(float(loss.detach().numpy()))
    accuracy_array.append(float(accuracy.numpy()))

    outputs_test = model(image_test)
    loss_test = criterion(outputs_test, test_labels)

    pred_y_test = torch.max(outputs_test, 1)[1].data.squeeze()
    accuracy_test = torch.sum((pred_y_test == torch.max(test_labels, 1)[1].
    ↪data.squeeze()) / pred_y_test.size(0))
    accuracy_test_array.append(float(accuracy_test.numpy()))

```

```

        loss_test_array.append(float(loss_test.detach().numpy()))

    if epoch % 50 == 0:
        print(epoch, accuracy.numpy(), loss.detach().numpy(), accuracy_test.
        numpy(), loss_test.detach().numpy())

```

```

0 0.5642168 2.0132668 0.56222224 1.7164289
50 0.98737955 0.027021453 0.9777776 0.10467865

```

```

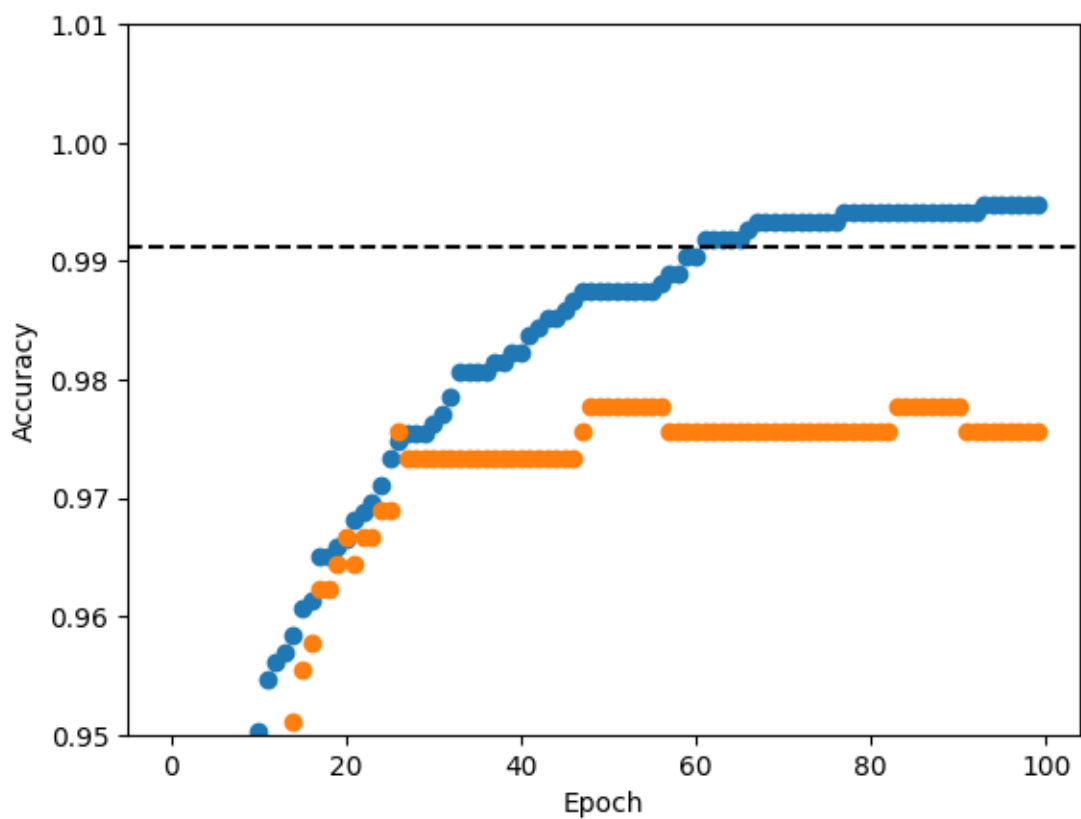
[55]: plt.scatter(epoch_array, accuracy_array, color='C0')
plt.scatter(epoch_array, accuracy_test_array, color='C1')
plt.ylim(0.95, 1.01)
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.axhline(0.9912, ls='--', color='black')

```

```

[55]: <matplotlib.lines.Line2D at 0x7a5519003be0>

```



The accuracy of the test dataset needs to be higher than the black dashed line.

```
[ ]: # Improved CNN architecture for higher accuracy
class CNN(torch.nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.cnn1 = torch.nn.Sequential(
            torch.nn.Conv2d(1, 32, kernel_size=3, padding=1),
            torch.nn.ReLU(),
            torch.nn.BatchNorm2d(32),
            torch.nn.MaxPool2d(2, 2),
            torch.nn.Conv2d(32, 64, kernel_size=3, padding=1),
            torch.nn.ReLU(),
            torch.nn.BatchNorm2d(64),
            torch.nn.MaxPool2d(2, 2),
            torch.nn.Conv2d(64, 128, kernel_size=3, padding=1),
            torch.nn.ReLU(),
            torch.nn.BatchNorm2d(128),
            torch.nn.MaxPool2d(2, 2),

        )
        self.fc1 = torch.nn.Sequential(
            torch.nn.Linear(128 * 1 * 1, 512),
            torch.nn.ReLU(),
            torch.nn.Dropout(0.5),
            torch.nn.Linear(512, 10)
        )

    def forward(self, x):
        x = self.cnn1(x)
        x = x.view(x.size(0), -1) # Flatten the tensor
        x = self.fc1(x)
        return x

model = CNN()
learningRate = 0.01
epochs = 200
criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learningRate)
N_total_train = len(image_train)
batch_size = 64

accuracy_array = []
accuracy_test_array = []
loss_array = []
loss_test_array = []
```

```

epoch_array = []

for epoch in range(epochs):
    model.train()

    for start_index_batch in range(0, N_total_train, batch_size):
        optimizer.zero_grad()

        end_index = min(start_index_batch + batch_size, N_total_train)
        inputs = image_train[start_index_batch:end_index]
        targets = image_labels[start_index_batch:end_index]

        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()

    model.eval()

    with torch.no_grad():

        outputs_all = model(image_train)
        pred_y = torch.max(outputs_all, 1)[1]
        accuracy_train = (pred_y == torch.max(image_labels, 1)[1]).sum().item()
        ↪ / pred_y.size(0)
        accuracy_array.append(accuracy_train * 100)
        loss_array.append(float(loss.item()))

        outputs_test = model(image_test)
        loss_test = criterion(outputs_test, test_labels)
        pred_y_test = torch.max(outputs_test, 1)[1]
        accuracy_test = (pred_y_test == torch.max(test_labels, 1)[1]).sum().
        ↪ item() / pred_y_test.size(0)
        accuracy_test_array.append(accuracy_test * 100)
        loss_test_array.append(float(loss_test.item()))

    epoch_array.append(epoch)

    # Print results
    if epoch % 50 == 0:
        print(f"Epoch {epoch}: Accuracy={accuracy_train * 100:.2f}%, Loss={loss.
        ↪ item():.4f}, Test Accuracy={accuracy_test * 100:.2f}%, Test Loss={loss_test.
        ↪ item():.4f}")

```

Epoch 0: Accuracy=91.31%, Loss=1.5590, Test Accuracy=90.67%, Test Loss=0.2783

Epoch 50: Accuracy=99.85%, Loss=-0.0000, Test Accuracy=98.44%, Test Loss=0.1070
Epoch 100: Accuracy=99.93%, Loss=-0.0000, Test Accuracy=99.33%, Test Loss=0.0688
Epoch 150: Accuracy=100.00%, Loss=-0.0000, Test Accuracy=99.11%, Test
Loss=0.1144

```
[57]: plt.scatter(epoch_array, accuracy_array, label='Training Accuracy')  
plt.scatter(epoch_array, accuracy_test_array, label='Test Accuracy')  
plt.xlabel('Epoch')  
plt.ylabel('Accuracy(%)')  
plt.axhline(99,ls='--',color='black')  
plt.legend()  
plt.show()
```

